Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Богатырева Е.Д.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 21.10.26

Москва, 2026

Постановка задачи

Вариант 2.

Отсортировать массив целых чисел при помощи параллельного алгоритма быстрой сортировки.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно задаваться ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Общий метод и алгоритм решения

Программа реализована на языке Си для ОС Linux (запускалась в WSL). Для создания потоков использована библиотека POSIX threads (pthreads), то есть стандартное средство Unix-систем; C11 <threads.h> не применялась. Остальная стандартная библиотека Си (stdio, stdlib, string и т.д.) использовалась свободно: по уточнению преподавателя ограничение касается только средств многопоточности <threads.h> и <thread>.

Использованные функции и системные вызовы:

- int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg) — создаёт поток, выполняющий функцию start(arg).
- int pthread_join(pthread_t thread, void **retval) — ожидает завершения потока.
- int pthread_mutex_lock / pthread_mutex_unlock — захват и освобождение мьютекса, защищающего счётчик активных потоков.
- int clock_gettime(CLOCK_MONOTONIC, struct timespec *ts) — замер времени сортировки.
- int getopt(int argc, char *const argv[], const char *optstring) — разбор ключей запуска (-t, -n, -m, -s, -r, -w).
- pid_t getpid(void), unsigned sleep(unsigned sec) — вывод PID и пауза для наблюдения за потоками системными утилитами.
- void *malloc(size_t) / void free(void *) — выделение памяти под массив и описание задачи потока.

Алгоритм работы программы:

- Ключом -t N задаётся максимальное число одновременно работающих потоков (включая главный). Дополнительные ключи: -n (размер массива), -m (тип данных: random, sorted, reverse, equal), -s (начальное значение генератора), -r (число повторов), -w (пауза перед выходом).
- Массив заполняется псевдослучайными числами (генератор xorshift64*) либо по шаблону выбранного типа.
- Сортировка выполняется функцией qsort_par. Если участок содержит не более 32 элементов, он сортируется вставками. Иначе выполняется разбиение по Хоару, опорный элемент выбирается как медиана трёх (первый, средний, последний). Получаются две непересекающиеся части, которые можно сортировать независимо, поэтому гонок по данным нет.

- Если участок не меньше PAR_CUTOFF = 50000 элементов и есть свободный слот потока (функция try_acquire), левая часть отдаётся новому потоку (pthread_create), а правую текущий поток сортирует сам, после чего вызывает pthread_join. Если слотов нет или участок мал, обе части сортируются последовательно: рекурсия идёт в меньшую часть, а большая обрабатывается циклом, поэтому глубина стека составляет O(log n).
- Ограничение числа потоков реализовано счётчиком active, который защищён мьютексом. Главный поток занимает один слот, новый поток занимает слот при создании и освобождает его при завершении. Таким образом, число одновременно существующих потоков никогда не превышает значение -t. Максимум фиксируется в переменной peak и выводится после каждого прогона.
- Пул потоков не использовался: у него выше накладные расходы на очередь задач и синхронизацию, а дерево потоков естественно повторяет дерево рекурсии быстрой сортировки.
- После сортировки результат проверяется функцией is_sorted. Время прогона измеряется через clock_gettime, итоговое значение — среднее по -r прогонам (в протоколе использовано 5 прогонов).

Демонстрация числа потоков.

Во время работы программы число потоков показывают стандартные утилиты Linux: ps -o pid,nlwp,cmd -p PID (столбец NLWP — число легковесных процессов, то есть потоков) и top -H -p PID (потоки по отдельности).

Код программы

psort.c

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>


#define INSERTION_CUTOFF 32
#define PAR_CUTOFF 50000


static int max_threads = 1;
static int active = 1;
static int peak = 1;
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;


static int try_acquire(void)
{
    int ok = 0;
    pthread_mutex_lock(&mtx);
    if (active < max_threads) {
        active++;
        if (active > peak) peak = active;
        ok = 1;
    }
    pthread_mutex_unlock(&mtx);
    return ok;
}


static void release(void)
{
    pthread_mutex_lock(&mtx);
    active--;
    pthread_mutex_unlock(&mtx);
}


static void reset_counters(void)
{
    pthread_mutex_lock(&mtx);
    active = 1;
    peak = 1;
    pthread_mutex_unlock(&mtx);
}


static inline void swap_int(int *x, int *y)
{
    int t = *x; *x = *y; *y = t;
}


static void insertion_sort(int *a, long lo, long hi)
{
    for (long i = lo + 1; i <= hi; i++) {
        int v = a[i];
        long j = i - 1;
        while (j >= lo && a[j] > v) { a[j + 1] = a[j]; j--; }
        a[j + 1] = v;
    }
}


static long partition(int *a, long lo, long hi)
{
    long mid = lo + (hi - lo) / 2;
```

```
    if (a[lo] > a[mid]) swap_int(&a[lo], &a[mid]);
    if (a[lo] > a[hi])  swap_int(&a[lo], &a[hi]);
    if (a[mid] > a[hi]) swap_int(&a[mid], &a[hi]);
    int p = a[mid];
    long i = lo - 1, j = hi + 1;
    for (;;) {
        do i++; while (a[i] < p);
        do j--; while (a[j] > p);
        if (i >= j) return j;
        swap_int(&a[i], &a[j]);
    }
}

typedef struct { int *a; long lo, hi; } task_t;

static void qsort_par(int *a, long lo, long hi);

static void *thread_main(void *arg)
{
    task_t *t = (task_t *)arg;
    int *a = t->a;
    long lo = t->lo, hi = t->hi;
    free(t);
    qsort_par(a, lo, hi);
    release();
    return NULL;
}

static void qsort_par(int *a, long lo, long hi)
{
    while (lo < hi) {
        if (hi - lo + 1 <= INSERTION_CUTOFF) {
            insertion_sort(a, lo, hi);
            return;
        }
        long j = partition(a, lo, hi);

        if (hi - lo + 1 >= PAR_CUTOFF && try_acquire()) {
            task_t *t = malloc(sizeof *t);
            if (t) {
                t->a = a; t->lo = lo; t->hi = j;
                pthread_t th;
                int rc = pthread_create(&th, NULL, thread_main, t);
                if (rc == 0) {
                    qsort_par(a, j + 1, hi);
                    pthread_join(th, NULL);
                    return;
                }
                free(t);
            }
            release();
        }
        if (j - lo < hi - j) { qsort_par(a, lo, j); lo = j + 1; }
        else                 { qsort_par(a, j + 1, hi); hi = j; }
    }
}

static double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

static unsigned long long rng_state;
static unsigned long long rng(void)
{
    rng_state ^= rng_state >> 12;
    rng_state ^= rng_state << 25;
```

```
    rng_state ^= rng_state >> 27;
    return rng_state * 2685821657736338717ULL;
}

static void fill(int *a, long n, const char *mode)
{
    for (long i = 0; i < n; i++) {
        if (!strcmp(mode, "sorted"))       a[i] = (int)(i % INT_MAX);
        else if (!strcmp(mode, "reverse")) a[i] = (int)((n - i) % INT_MAX);
        else if (!strcmp(mode, "equal"))   a[i] = 42;
        else                               a[i] = (int)(rng() >> 33);
    }
}

static int is_sorted(const int *a, long n)
{
    for (long i = 1; i < n; i++)
        if (a[i - 1] > a[i]) return 0;
    return 1;
}

int main(int argc, char **argv)
{
    long n = 10000000;
    int repeats = 3, wait_sec = 0;
    unsigned long long seed = 12345;
    const char *mode = "random";

    int opt;
    while ((opt = getopt(argc, argv, "t:n:s:m:r:w:")) != -1) {
        switch (opt) {
        case 't': max_threads = atoi(optarg); break;
        case 'n': n = atol(optarg); break;
        case 's': seed = strtoull(optarg, NULL, 10); break;
        case 'm': mode = optarg; break;
        case 'r': repeats = atoi(optarg); break;
        case 'w': wait_sec = atoi(optarg); break;
        default:
            fprintf(stderr,
                "Usage: %s -t threads -n size [-s seed] [-m random|sorted|reverse|equal] [-r
repeats] [-w sec]\n",
                argv[0]);
            return 1;
        }
    }
    if (max_threads < 1 || n < 1 || repeats < 1) {
        fprintf(stderr, "Некорректные параметры\n");
        return 1;
    }
    if (n > (long)INT_MAX) {
        fprintf(stderr, "n слишком велико для int-элементов: максимум %d\n", INT_MAX);
        return 1;
    }

    int *a = malloc((size_t)n * sizeof(int));
    if (!a) { perror("malloc"); return 1; }

    printf("PID=%d, max_threads=%d, n=%ld, mode=%s\n", getpid(), max_threads, n, mode);
    fflush(stdout);

    double total = 0;
    for (int r = 0; r < repeats; r++) {
        rng_state = seed + (unsigned long long)r;
        fill(a, n, mode);
        reset_counters();

        double t0 = now();
        qsort_par(a, 0, n - 1);
```

```
        double dt = now() - t0;

        if (!is_sorted(a, n)) {
            fprintf(stderr, "ОШИБКА: массив не отсортирован\n");
            free(a);
            return 2;
        }
        printf("run %d: %.4f s, пик потоков = %d\n", r + 1, dt, peak);
        total += dt;
    }
    printf("RESULT threads=%d n=%ld mode=%s avg_time=%.6f\n",
           max_threads, n, mode, total / repeats);

    if (wait_sec > 0) { /* для демонстрации: ps -o pid,nlwp,cmd -p PID; top -H -p PID */
        printf("Пауза %d с, PID=%d...\n", wait_sec, getpid());
        fflush(stdout);
        sleep(wait_sec);
    }
    free(a);
    return 0;
}
```

Протокол работы программы

Условия эксперимента:

- среда: Linux в WSL на Windows, nproc = 16 (логических процессоров);
- сборка: CMake (CLion), gcc; программа OS_lab2;
- массив типа int, значения заполняются генератором со стартовым значением 12345;
- для каждой точки выполнялось 5 прогонов (-r 5), в таблицах приведено среднее время сортировки (без заполнения массива и проверки).

Тест 1. Проверка работы и ограничения потоков (n = 10^6, -t 4):

```
$ ./OS_lab2 -t 4 -n 1000000
PID=4788, max_threads=4, n=1000000, mode=random
run 1: 0.0659 s, пик потоков = 4
run 2: 0.0687 s, пик потоков = 4
run 3: 0.0632 s, пик потоков = 4
RESULT threads=4 n=1000000 mode=random avg_time=0.065940
```

Тест 2. Зависимость от числа потоков (n = 10^7, random):

```
$ for t in 1 2 4 8 16; do ./OS_lab2 -t $t -n 10000000 -r 5; done
RESULT threads=1  n=10000000 mode=random avg_time=1.787386
RESULT threads=2  n=10000000 mode=random avg_time=1.290899
RESULT threads=4  n=10000000 mode=random avg_time=0.821648
RESULT threads=8  n=10000000 mode=random avg_time=0.491312
RESULT threads=16 n=10000000 mode=random avg_time=0.348662
```

Таблица 1. Время, ускорение и эффективность (n = 10^7).

p	T(p), с	S = T(1)/T(p)	E = S/p, %	S (модель)
1	1.787	1.00	100	1.00
2	1.291	1.38	69	1.92
4	0.822	2.18	54	3.41
8	0.491	3.64	45	5.43
16	0.349	5.13	32	7.55

Тест 3. Зависимость от типа входных данных (n = 10^7, -t 8):

```
$ ./OS_lab2 -t 8 -n 10000000 -m sorted -r 3
RESULT threads=8 n=10000000 mode=sorted  avg_time=0.077682
$ ./OS_lab2 -t 8 -n 10000000 -m reverse -r 3
RESULT threads=8 n=10000000 mode=reverse avg_time=0.108911
$ ./OS_lab2 -t 8 -n 10000000 -m equal -r 3
RESULT threads=8 n=10000000 mode=equal   avg_time=0.219822
```

Таблица 2. Время сортировки при разных типах данных (8 потоков).

Тип данных	T(8), с	Во сколько раз быстрее random
random	0.491	1.0
sorted	0.078	6.3
reverse	0.109	4.5
equal	0.220	2.2

Тест 4. Большой массив (n = 10^8, -t 8):

```
$ ./OS_lab2 -t 8 -n 100000000 -r 5
run 1: 2.4278 s, пик потоков = 8
run 2: 2.6624 s, пик потоков = 8
run 3: 2.5967 s, пик потоков = 8
run 4: 2.5029 s, пик потоков = 8
run 5: 2.6743 s, пик потоков = 8
RESULT threads=8 n=100000000 mode=random avg_time=2.572818
```

Тест 5. Демонстрация числа потоков средствами ОС (n = 10^8, -t 8):

```
$ ./OS_lab2 -t 8 -n 100000000 -r 5 &
$ PID=$!
$ while kill -0 $PID 2>/dev/null; do ps -o pid,nlwp,cmd -p $PID | tail -1; sleep 0.3;
done
  12458    1 ./OS_lab2 -t 8 -n 100000000 -r 5      <- заполнение массива, один поток
  12458    1 ./OS_lab2 -t 8 -n 100000000 -r 5
  12458    2 ./OS_lab2 -t 8 -n 100000000 -r 5      <- начало сортировки
  12458    8 ./OS_lab2 -t 8 -n 100000000 -r 5      <- достигнут лимит
  12458    8 ./OS_lab2 -t 8 -n 100000000 -r 5
  12458    8 ./OS_lab2 -t 8 -n 100000000 -r 5
  12458    6 ./OS_lab2 -t 8 -n 100000000 -r 5      <- часть потоков завершилась
  12458    8 ./OS_lab2 -t 8 -n 100000000 -r 5
  12458    1 ./OS_lab2 -t 8 -n 100000000 -r 5      <- проверка результата, один поток
run 1: 2.3666 s, пик потоков = 8
```

Значение NLWP ни разу не превысило 8, что совпадает с ключом -t 8 и с выводом самой программы («пик потоков = 8»). При разовом снимке через 3 секунды после запуска ps показал NLWP = 7: часть потоков к этому моменту уже завершилась.

Анализ результатов.

1. Зависимость от числа потоков. Время падает монотонно, при 16 потоках сортировка выполняется в 5.1 раза быстрее, чем при одном (таблица 1). Ускорение растёт, но заметно медленнее идеального S = p, поэтому эффективность убывает с 69% (2 потока) до 32% (16 потоков).

2. Почему ускорение сублинейное. Первое разбиение выполняет один поток по всему массиву, на втором уровне рекурсии работает не более двух потоков, на третьем — не более четырёх и так далее. Параллелизм «раскручивается» постепенно. Для p = 2^m потоков и идеально ровных разбиений время пропорционально $n\cdot(1 + 1/2 + \ldots + 1/2^{(m-1)}) + n\cdot(\log_2 n - m)/p$. Это даёт модельное ускорение 1.9, 3.4, 5.4 и 7.5 для 2, 4, 8 и 16 потоков (столбец «S (модель)»). Измеренные значения составляют около 65–70% от модельных. Причины расхождения:

- разбиения неравные. Медиана трёх не гарантирует деления пополам, поток с большей частью работает дольше, остальные завершаются раньше. Это видно по ps: NLWP временами опускается до 3–7, хотя лимит равен 8;
- слот потока остаётся занятым, пока родитель ждёт pthread_join, поэтому одновременно работающих потоков фактически меньше значения -t;
- разбиение — это проход по массиву из 40 МБ (10^7 значений int), который не помещается в кэш. Потоки делят пропускную способность памяти, и операция упирается не в вычисления, а в обмен с памятью;
- накладные расходы на создание потоков, синхронизацию на мьютексе и работу планировщика ОС (в WSL поверх Windows дополнительно сказывается фоновая нагрузка).

3. Эффективность. E = S/p убывает, потому что неизбежно последовательная часть (верхние уровни рекурсии) и затраты на синхронизацию составляют всё большую долю времени, а каждый дополнительный поток добавляет меньше полезной работы (закон Амдала).

4. Число потоков и число ядер. Компьютер имеет 16 логических процессоров. Возможно, это меньшее число физических ядер с технологией SMT (в работе это не проверялось). При p = 16 прирост по сравнению с p = 8 составил 1.41 раза против 1.67 раза при переходе от 4 к 8, то есть отдача уже падает. При p, превышающем число логических процессоров, прироста ожидать не следует: потоки конкурируют за процессоры, а переключение контекста добавляет накладные расходы.

5. Зависимость от типа данных. Отсортированные и обратно упорядоченные массивы сортируются в 6.3 и 4.5 раза быстрее случайного (таблица 2). Для них медиана трёх выбирает точную середину, поэтому разбиения получаются ровными, а ветвления в циклах Хоара предсказываются процессором почти идеально. На случайных данных результат сравнения с опорным элементом непредсказуем, и процессор постоянно ошибается в предсказании ветвлений. Массив из одинаковых элементов сортируется дольше упорядоченного (на каждом шаге выполняется обмен), но разбиения всё равно получаются ровными, а алгоритм Хоара не вырождается в квадратичный.

6. Зависимость от размера массива. При четырёх потоках сортировка 10^6 элементов заняла 0.066 с, а 10^7 элементов — 0.822 с: время выросло в 12.5 раза при росте размера в 10 раз, что близко к ожидаемым n·log n (≈11.5 раза). При восьми потоках 10^7 элементов сортируются за 0.491 с, а 10^8 — за 2.57 с, то есть время выросло лишь в 5.2 раза. Это означает, что с ростом n относительная доля накладных расходов и разгона параллелизма уменьшается, а ускорение улучшается. Для количественной оценки ускорения на 10^8 необходим однопоточный прогон на тех же данных.

Вывод

В ходе лабораторной работы получены практические навыки управления потоками в ОС Linux и обеспечения их синхронизации. Разработана программа на Си, выполняющая параллельную быструю сортировку массива целых чисел с помощью POSIX threads. Максимальное число одновременно работающих потоков задаётся ключом запуска -t и контролируется счётчиком, защищённым мьютексом. Число потоков подтверждено стандартными средствами ОС (ps, столбец NLWP): оно не превышает заданного значения.

Исследование показало, что для массива из 10^7 элементов при 16 потоках достигнуто ускорение 5.1 раза при эффективности 32%. Ускорение сублинейное: ему мешают последовательное первое разбиение, неравномерность разбиений, ограниченная пропускная способность памяти и накладные расходы на потоки. Время сортировки сильно зависит и от типа данных: упорядоченные массивы сортируются в разы быстрее случайных. Использование числа потоков больше числа логических процессоров нецелесообразно.